

ShipPulse

Startup-Style Azure DevOps Assignment (Beginner-Friendly, No Assumptions)

Version 2.0 | Date: March 02, 2026

Includes: Backend (App Service), Frontend (Static Web Apps), VM (Nginx + optional self-hosted runner)

How to Use This Handout

This is written in a KodeKloud-style format: small tasks, exact names, exact expected results.

You will use a starter repo (provided as a zip) so you don't waste time scaffolding. Your job is to implement DevOps step-by-step.

You must not assume tools are installed on your laptop. Use Azure Cloud Shell for Azure CLI/Bicep work.

Deliverables are verified by screenshots + pipeline runs.

Tools You Are Allowed to Use (Office Laptop Constraint)

Local machine:

- Git
- .NET SDK (already installed)

Azure work:

- Azure Portal
- Azure Cloud Shell (Azure CLI + Bicep included)
- GitHub Actions

What You Already Know (We Will Reuse)

You already practiced these, and this assignment will use them for real:

- Azure VM creation + start/stop, SSH IP whitelisting, opening port 80
- Installing Nginx on Ubuntu
- Bicep/ARM basics: build/lint/validate/what-if/deploy
- Service Principal usage in GitHub Actions
- Azure App Service basic deployment
- Azure Static Web Apps basic deployment
- GitHub PRs + basic branch protection

Starter Repo (No Assumptions)

You will start from a working repository that contains:

- Backend (.NET 8 Minimal API) with /health and /api/feedback endpoints
- Frontend (Blazor WebAssembly) that calls the backend
- IaC (Bicep) for App Service + Key Vault + App Insights + Log Analytics + optional Jumpbox VM
- GitHub Actions workflows: CI, Deploy Backend, Deploy Frontend

Trainer will provide you a zip named: ShipPulse_StarterRepo.zip

Your first task is to create a private repo in the provided GitHub org and push this code.

Infrastructure Details (Fixed Names)

Use these exact names so the trainer can verify easily. Some resources require global uniqueness; follow the suffix rules below.

Set ONE suffix and reuse it everywhere: UNIQUE_SUFFIX = demo01 (example). Keep it short.

Env	Resource	Name Pattern	Resource Group	Region	SKU / Notes
Dev	Resource Group	rg-shippulse-dev	rg-shippulse-dev	centralindia	-
Prod	Resource Group	rg-shippulse-prod	rg-shippulse-prod	centralindia	-
Dev	App Service Plan	asp-shippulse-dev-<suffix>	rg-shippulse-dev	centralindia	B1
Prod	App Service Plan	asp-shippulse-prod-<suffix>	rg-shippulse-prod	centralindia	B1 (or P1v3 if allowed)
Dev	Web App (API)	app-shippulse-api-dev-<suffix>	rg-shippulse-dev	centralindia	Linux, .NET 8
Prod	Web App (API)	app-shippulse-api-prod-<suffix>	rg-shippulse-prod	centralindia	Linux, .NET 8
Dev	Key Vault	kvshippulsedev<suffix>	rg-shippulse-dev	centralindia	Global unique, letters+numbers only
Prod	Key Vault	kvshippulseprod<suffix>	rg-shippulse-prod	centralindia	Global unique, letters+numbers only

					ers only
Dev	App Insights	appi-shippulse-dev-<suffix>	rg-shippulse-dev	centralindia	Workspace-based
Prod	App Insights	appi-shippulse-prod-<suffix>	rg-shippulse-prod	centralindia	Workspace-based
Dev	Log Analytics	law-shippulse-dev-<suffix>	rg-shippulse-dev	centralindia	Retention 30 days
Prod	Log Analytics	law-shippulse-prod-<suffix>	rg-shippulse-prod	centralindia	Retention 30 days
Dev	Ubuntu VM (Jumpbox)	vm-shippulse-jump-<suffix>	rg-shippulse-dev	centralindia	B1ms, SSH from your IP only
Dev	Nginx on VM	nginx default site	vm-shippulse-jump-<suffix>	centralindia	Port 80 open
Dev	Static Web App	swa-shippulse-dev-<suffix>	rg-shippulse-dev	centralindia	Create via Portal; token in GitHub secret
Prod	Static Web App	swa-shippulse-prod-<suffix>	rg-shippulse-prod	centralindia	Create via Portal; token in GitHub secret

Reference: Azure Cloud Shell comes with Azure CLI and Bicep preinstalled. You must use Cloud Shell if you cannot install tools locally.

<https://learn.microsoft.com/en-us/azure/cloud-shell/features>

Day 0 - Setup (Must Finish Before Day 1)

TASK 0.1 - Create the GitHub repository

Requirement:

- Create a private repo in your training org named: shippulse
- Create branches: main and develop
- Enable branch protection on main: require PR, require 1 approval

Expected result:

- Repo exists in org
- main and develop branches exist
- PR required for main merges

TASK 0.2 - Push starter code

Instructions:

- Download ShipPulse_StarterRepo.zip from trainer and unzip
- In local terminal:
 - git init
 - git remote add origin <YOUR_REPO_URL>
 - git checkout -b develop
 - git add . && git commit -m "chore: initial starter"
 - git push -u origin develop
- Create main branch from develop and push

Validation:

- GitHub shows files: src/, infra/, .github/workflows/
- Actions tab shows workflows present (they may not run yet)

TASK 0.3 - Azure subscription access check

Open Azure Portal -> Cloud Shell -> Bash. Run:

```
az account show
az account list --output table
```

Expected result: you can see subscription name/id and tenant. If you cannot create resources, ask trainer for permissions.

Day 1 - Run the Product Locally + First PR

TASK 1.1 - Run backend locally

Run:

```
dotnet restore
dotnet run --project src/ShipPulse.Api
```

Expected result:

- GET <http://localhost:5080/health> returns status ok
- Swagger UI is available in Development at </swagger>

TASK 1.2 - Run frontend locally

Run (in a new terminal):

```
dotnet run --project src/ShipPulse.Web
```

Expected result:

- Frontend loads at <http://localhost:5170>
- Submitting feedback adds an item and refreshes list

Hint: If frontend cannot reach backend, ensure backend is running and CORS allows <http://localhost:5170> (already set in backend Development config).

TASK 1.3 - Make a small change and open a PR

Change the site title in `src/ShipPulse.Web/Shared/MainLayout.razor` (example: add your team name).

Create a feature branch and PR into develop.

Expected result: CI should run on PR and pass (or fail with clear error you fix).

Day 1 Questions (write answers in your notes)

- Why do we keep main protected and merge via PR?
- What is the difference between CI and CD in this project?
- Why is Swagger enabled only in Development?

Day 2 - VM + Nginx (Use What You Learned)

TASK 2.1 - Create dev resource group

In Azure Cloud Shell (Bash):

```
az group create -n rg-shippulse-dev -l centralindia
az group create -n rg-shippulse-prod -l centralindia
```

TASK 2.2 - Deploy dev infra (includes VM) using Bicep

You must edit infra/env/dev.bicepparam first to set:

- jumpboxSshPublicKey
- sshAllowedCidr (your public IP/32)

Hint: Generate SSH key in Cloud Shell:

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/shippulse_id_rsa -N ""
cat ~/.ssh/shippulse_id_rsa.pub
```

Find your public IP (example): <https://ifconfig.me> and use /32. Example: 1.2.3.4/32

Deploy (Cloud Shell):

```
az deployment group validate -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
az deployment group what-if -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
az deployment group create -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
```

Expected result: VM + Web App + Key Vault + App Insights + Log Analytics are created in rg-shippulse-dev.

TASK 2.3 - Install Nginx on the VM

SSH from local machine (Windows PowerShell or Git Bash):

```
ssh -i <PATH_TO_PRIVATE_KEY> azureuser@<JUMPBOX_PUBLIC_IP>
```

On VM:

```
sudo apt-get update
sudo apt-get install -y nginx
sudo systemctl enable --now nginx
curl -I http://localhost
```

Expected result: browsing http://<JUMPBOX_PUBLIC_IP> shows Nginx welcome page.

Hint: If you cannot access port 80, verify NSG inbound rule Allow-HTTP-80 exists (created by Bicep).

Day 3 - CI Improvements + Deploy Backend to App Service (Dev)

TASK 3.1 - Configure GitHub secrets for backend deployment

Create GitHub Environments: dev and prod (Repo settings -> Environments).

Add secrets (dev environment):

- AZURE_CLIENT_ID
- AZURE_TENANT_ID
- AZURE_SUBSCRIPTION_ID
- AZURE_RG_DEV = rg-shippulse-dev
- AZURE_WEBAPP_NAME_DEV = app-shippulse-api-dev-<suffix>

Add secrets (prod environment):

- AZURE_RG_PROD = rg-shippulse-prod
- AZURE_WEBAPP_NAME_PROD = app-shippulse-api-prod-<suffix>

Hint: Start with Service Principal secret-based auth if OIDC is not ready; upgrade to OIDC on Day 6/7.

TASK 3.2 - Run deploy-backend workflow to dev

Merge a PR into develop. The workflow deploy-backend.yml should run.

Expected result:

- Backend deployed to <https://<dev-webapp>.azurewebsites.net/health> returns ok

Screenshot required: GitHub Actions run summary + health endpoint in browser.

Day 3 Questions

- Why should deployment use the same artifact built in CI?
- Why do we deploy infrastructure from pipeline (IaC) and not only from portal?

Day 4 - Deploy Frontend to Static Web Apps (Dev + Prod)

TASK 4.1 - Create Static Web App (Dev) via Portal

Portal -> Create resource -> Static Web App.

- Name: swa-shippulse-dev-<suffix> (or similar unique name)
- Plan: Free
- Deployment: GitHub
- Branch: develop

Portal will offer to create a GitHub Action workflow automatically. Allow it, OR keep your existing deploy-frontend.yml and use the deployment token.

Get the deployment token: Static Web App -> Manage deployment token. Store as GitHub secret AZURE_STATIC_WEB_APPS_API_TOKEN_DEV.

Also set API_BASE_URL_DEV = https://<dev-webapp>.azurewebsites.net

TASK 4.2 - Create Static Web App (Prod) via Portal

- Name: swa-shippulse-prod-<suffix>
- Branch: main
- Token secret: AZURE_STATIC_WEB_APPS_API_TOKEN_PROD
- API_BASE_URL_PROD = https://<prod-webapp>.azurewebsites.net

TASK 4.3 - Deploy frontend

Push to develop to deploy dev frontend. Push/merge to main to deploy prod frontend.

Expected result:

- Dev frontend loads from Static Web Apps URL
- Submitting feedback from dev frontend reaches dev backend

Day 5 - Secrets: Key Vault + Managed Identity (Runtime)

TASK 5.1 - Create a Key Vault secret

In dev Key Vault, create secret:

Name: ExternalApi--ApiKey

Value: any demo value (e.g., DEMO-123)

Hint: Use the Key Vault created by Bicep in rg-shippulse-dev.

TASK 5.2 - Grant Web App access to Key Vault (RBAC)

Portal -> Key Vault -> Access control (IAM) -> Add role assignment:

- Role: Key Vault Secrets User
- Assign access to: Managed identity
- Select: your dev Web App

Expected result: Web App can resolve Key Vault reference.

TASK 5.3 - Verify Key Vault reference works

Portal -> Web App -> Configuration -> Application settings:

ExternalApi__ApiKey should already be set using Key Vault reference format.

Restart the Web App and verify no errors in logs.

Day 5 Questions

- What is the difference between deployment secrets and runtime secrets?
- Why is managed identity preferred over putting secrets in config files?

Day 6 - Monitoring + Runner Comparison

TASK 6.1 - Enable Application Insights

Portal -> Web App -> Application Insights -> Turn on (or confirm connected).

Generate traffic:

Open frontend and submit a few feedback entries.

Expected result: Requests appear in Application Insights -> Failures/Performance.

Screenshot required: Application Insights showing requests.

TASK 6.2 - Create an alert

Create one alert rule for prod Web App:

- Signal: Failed requests (or 5xx) above threshold
- Action group: email to trainer/team (or use default)

Expected result: alert rule exists (screenshot).

TASK 6.3 - GitHub-hosted vs Self-hosted runner (optional but recommended)

Install a self-hosted runner on the jumpbox VM and run a simple job on it.

Hint: GitHub repo -> Settings -> Actions -> Runners -> New self-hosted runner. Follow Ubuntu instructions.

Expected result: runner shows Online. You can run a workflow job with runs-on: self-hosted.

Write 5 bullet points comparing GitHub-hosted and self-hosted runners.

Day 7 - Production Polish + (Optional) OIDC Upgrade

TASK 7.1 - Hardening checklist

- ☐ Ensure httpsOnly = true on web apps
- ☐ Ensure minimum TLS = 1.2
- ☐ Ensure prod deploy only from main
- ☐ Enable manual approval for prod environment (if available in your plan)

TASK 7.2 - Write final docs (must submit)

- ☐ DECISIONS.md (why App Service, why SWA, why Key Vault, why monitoring, why runner choice)
- ☐ INTERVIEW_NOTES.md (a 2-minute story + 10 bullet points)

TASK 7.3 - Optional: Upgrade GitHub Actions auth to OIDC

If you currently use a Service Principal secret, upgrade to OIDC for short-lived auth.

Hint: Use Azure App Registration + Federated Credentials for your GitHub repo/branch/environment.

Reference: <https://learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect>

Final Submission Checklist

- ☐ CI passing on PRs
- ☐ Dev backend deployed and /health works
- ☐ Prod backend deployed and /health works
- ☐ Dev frontend deployed and can submit feedback
- ☐ Prod frontend deployed
- ☐ Key Vault secret created and referenced by Web App
- ☐ App Insights shows requests
- ☐ One alert rule created
- ☐ Screenshots folder or doc with evidence

References

- Azure Cloud Shell tools (Azure CLI + Bicep): <https://learn.microsoft.com/en-us/azure/cloud-shell/features>
- GitHub Environments: <https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>
- Deploy to App Service with GitHub Actions: <https://learn.microsoft.com/en-us/azure/app-service/deploy-github-actions>
- App Service Key Vault references:
<https://learn.microsoft.com/en-us/azure/app-service/app-service-key-vault-references>
- Codeless Application Insights for App Service:
<https://learn.microsoft.com/en-us/azure/azure-monitor/app/codeless-app-service>